

Bayesian Scaffold — Drop-in Middleware for Ollama Agents

Three files. One decision layer. Your agent stops being a yes-machine.

Files

File	Role
memory_store.py	Persistent JSON belief store — priors, bias fingerprint, outcome history
bayesian_middleware.py	Decision engine — classifies input, routes, builds system prompt
scaffold_example.py	Minimal working loop with Ollama
monitor.html	Live visualiser — drop in <code>scaffold_memory.json</code> to inspect

Drop-in Usage

```
from bayesian_middleware import BayesianMiddleware

mw = BayesianMiddleware(user_id="e404")

# Before Ollama call
decision = mw.pre_process(user_message)

# decision.route          → comply | reframe | clarify | challenge
# decision.bias_detected → anchoring | vague_intent | etc.
# decision.system_injection → enriched system prompt string

response = call_ollama(decision.system_injection, user_message, history)

# After Ollama responds
mw.post_process(response, decision)
```

→ updates priors, writes to scaffold_memory.json

Routes

Route	When	What the agent does
comply	Clear intent, high AI confidence	Produces exactly what was asked
reframe	Detectable intent beneath stated request	Produces the better answer, notes what it did
clarify	Insufficient signal	Asks one focused question
challenge	Flawed premise / cognitive bias detected	Surfaces the flaw, offers reframe

Assertiveness drift shifts the balance: the more often the AI gets wrong outcomes, the bolder it becomes — leaning toward `reframe` and `challenge` over `comply`.

Memory Store

`scaffold_memory.json` persists across sessions:

- `human_prior` — $P(\text{human knows what they want})$
 - `ai_prior` — $P(\text{AI can satisfy without clarification})$
 - `agent_assertiveness` — drifts $0 \rightarrow 0.90$ as wrong-outcome rate rises
 - `bias_counts` — fingerprint of user's cognitive patterns
 - `outcome_counts` — hit/clarify/wrong history
 - `recent` — rolling 10-entry window for trend detection
-

Monitor

Open `monitor.html` in any browser.

- Manual simulator works standalone (no Python needed)

- Drop `scaffold_memory.json` onto the upload zone to visualise a live session
 - Shows: belief posteriors, bias radar, outcome distribution, assertiveness dial, route decisions
-

Swap backends

`MemoryStore` uses JSON by default. To use SQLite:

```
import sqlite3
from memory_store import MemoryStore

class SQLiteStore(MemoryStore):
    def _load(self): ...    # read from SQLite
    def _save(self): ...    # write to SQLite
```

API is identical. Everything else just works.